Computational Neuroscience

Assignment (2)

By: Momen Tarek Gaber

4221022

Contents

1 ✓ Network Structure

2 ✓ Initial Weights

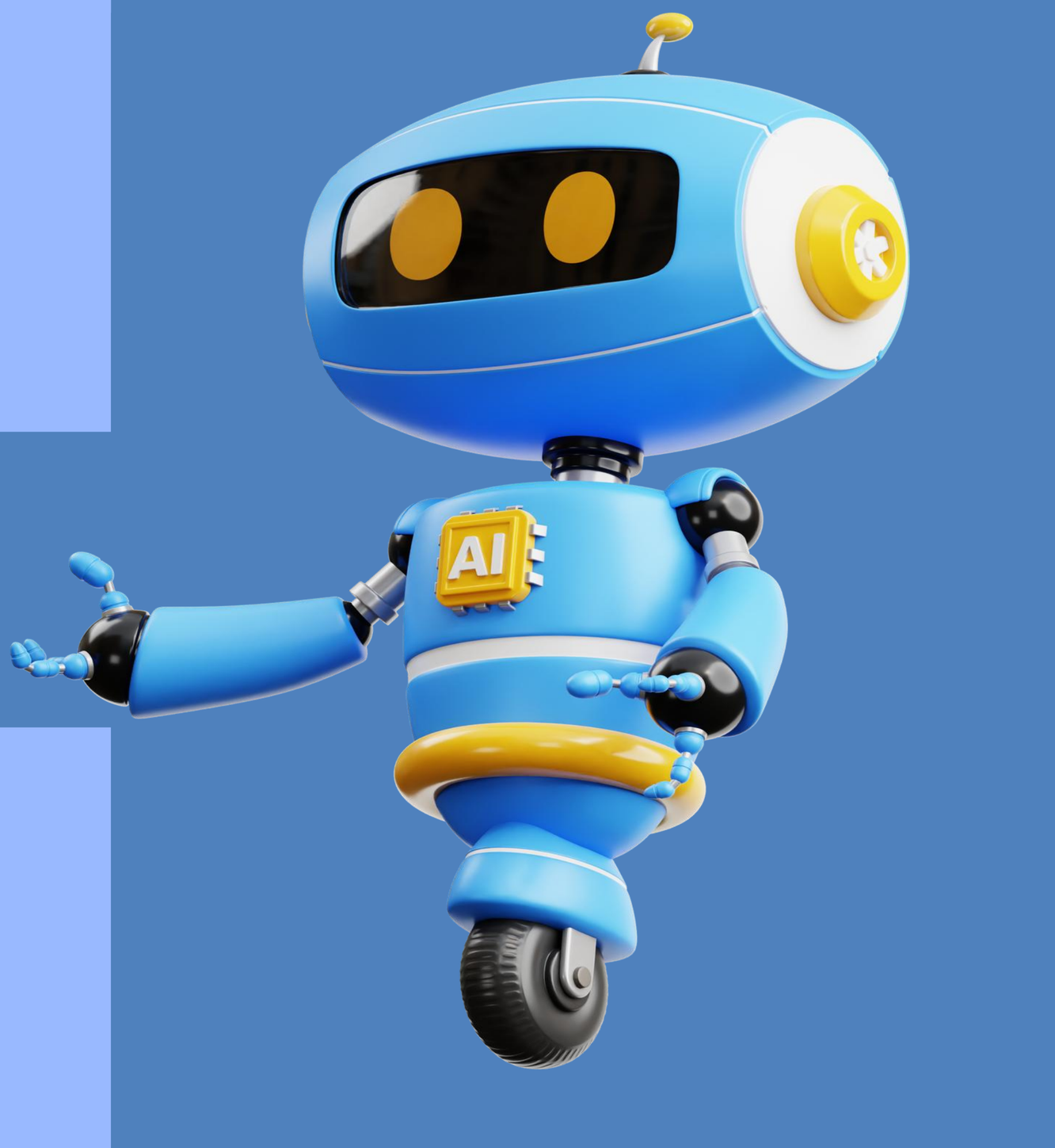
3 ✓ Activation Function

Network Structure

- Input Layer: 2 Neurons (x1,x2)

- Hidden Layer: 2 Neurons (h1,h2)

- Output Layer: 2 Neurons (o1,o2)

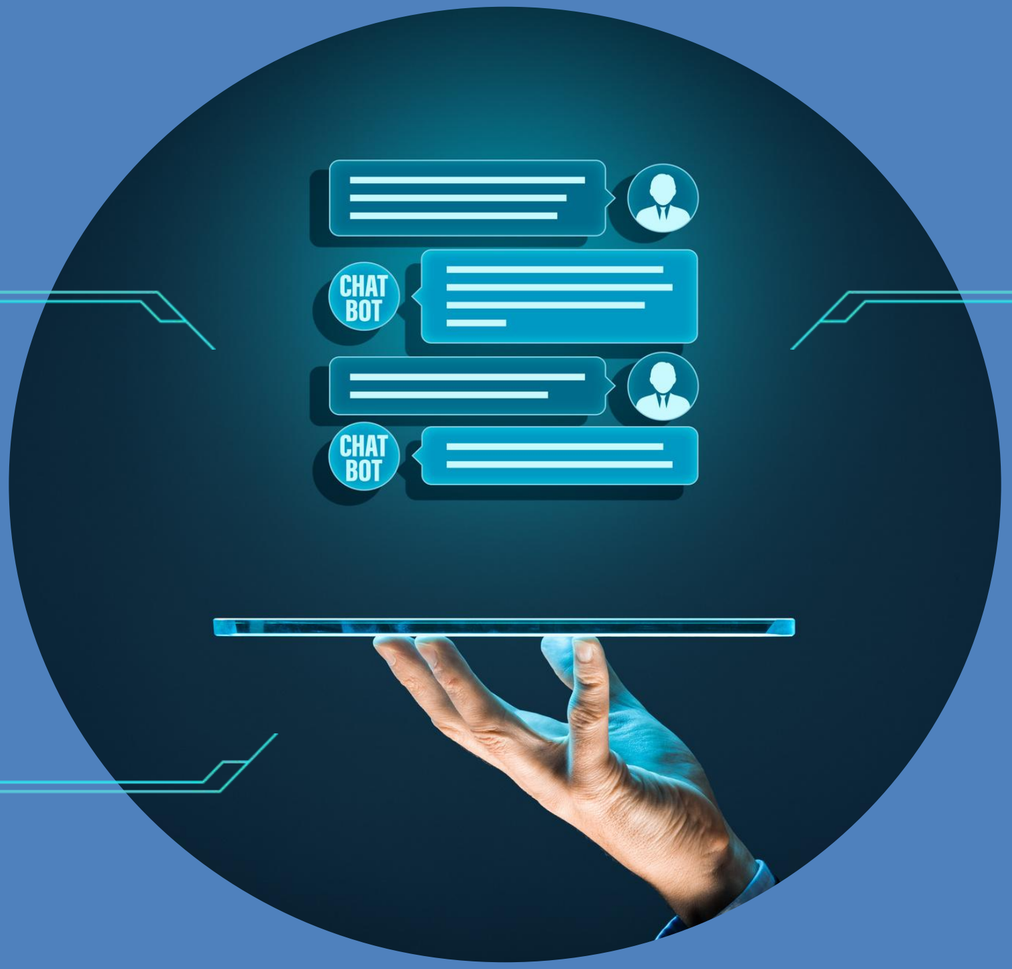


Initial Weights

✓ w6 and w7 as the weights from the hidden layer to the output layer, and w8 as a weight connecting respective inputs to hidden neurons

Activation Function

Step 1: Forward Propagation
Step 2: Calculate Error
Step 3: Backpropagation



Output: Finally, we print the trained output along with the updated weights.

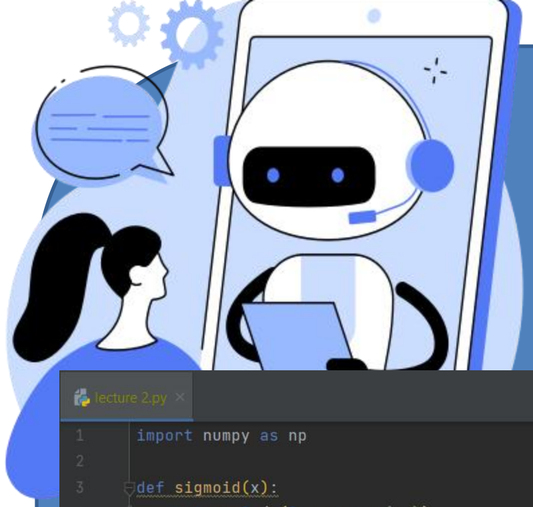
Training: The loop is run for 10,000 epochs to allow the model to learn.

without

libarary

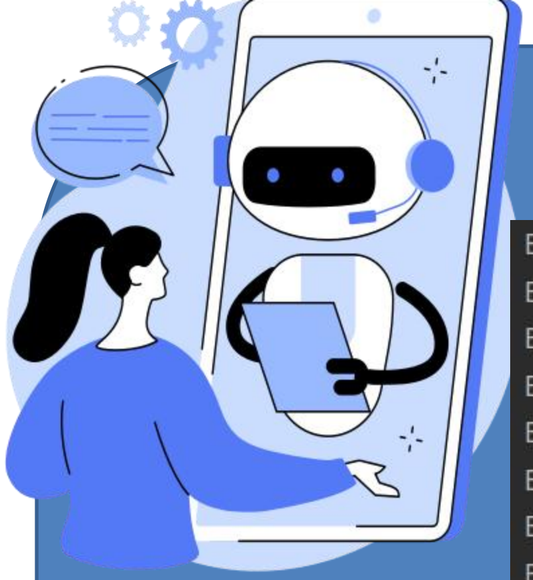
>>>>>>

Code



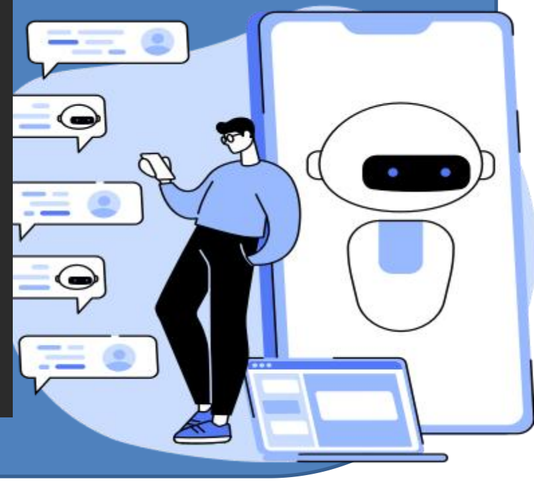
lecture 2.py

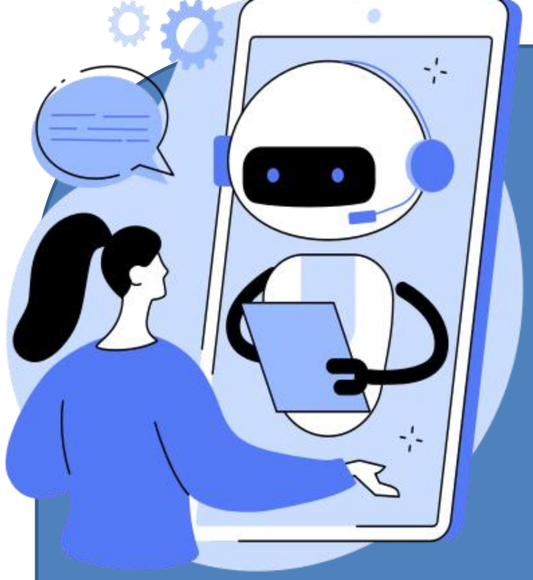
```python
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

inputs = np.array([[0, 0],
                   [0, 1],
                   [1, 0],
                   [1, 1]])
expected_output = np.array([[0],
                            [1],
                            [1],
                            [0]])
np.random.seed(42)
weights_input_hidden = np.random.uniform(size=(2, 2))
weights_hidden_output = np.random.uniform(size=(2, 1))

learning_rate = 0.5

for epoch in range(10000):
    hidden_layer_input = np.dot(inputs, weights_input_hidden)
    hidden_layer_output = sigmoid(hidden_layer_input)

    output_layer_input = np.dot(hidden_layer_output, weights_hidden_output)
    predicted_output = sigmoid(output_layer_input)

    error = expected_output - predicted_output
    d_predicted_output = error * sigmoid_derivative(predicted_output)

    weights_hidden_output += hidden_layer_output.T.dot(d_predicted_output) * learning_rate

    error_hidden_layer = d_predicted_output.dot(weights_hidden_output.T)
    d_hidden_layer = error_hidden_layer * sigmoid_derivative(hidden_layer_output)

    weights_input_hidden += inputs.T.dot(d_hidden_layer) * learning_rate
    if epoch % 1000 == 0:
        print(f"Epoch {epoch} Error: {np.mean(np.abs(error))}")

print("\nFinal Output after training:")
print(predicted_output)
print("Weights from Input to Hidden Layer:")
print(weights_input_hidden)
print("Weights from Hidden to Output Layer:")
print(weights_hidden_output)
```



Result

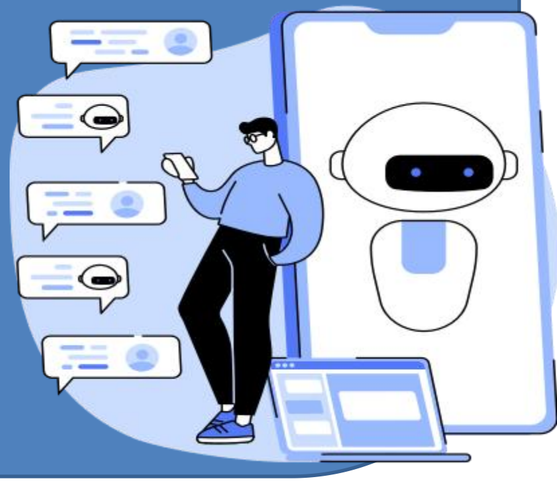
```
Epoch 0 Error: 0.4994268057883715
Epoch 1000 Error: 0.395139951051301
Epoch 2000 Error: 0.26879430368831964
Epoch 3000 Error: 0.20867649953638484
Epoch 4000 Error: 0.17382485833010863
Epoch 5000 Error: 0.15091089742238467
Epoch 6000 Error: 0.13457064556111836
Epoch 7000 Error: 0.12224813476527796
Epoch 8000 Error: 0.11257083204533688
Epoch 9000 Error: 0.104734323661926

Final Output after training:
[[0.05432236]
 [0.89824713]
 [0.89824729]
 [0.13513498]]
Weights from Input to Hidden Layer:
[[0.90572399 7.32512771]
 [0.90573083 7.32770127]]
Weights from Hidden to Output Layer:
[[-27.46456297]
 [ 21.75046845]]

Process finished with exit code 0
```

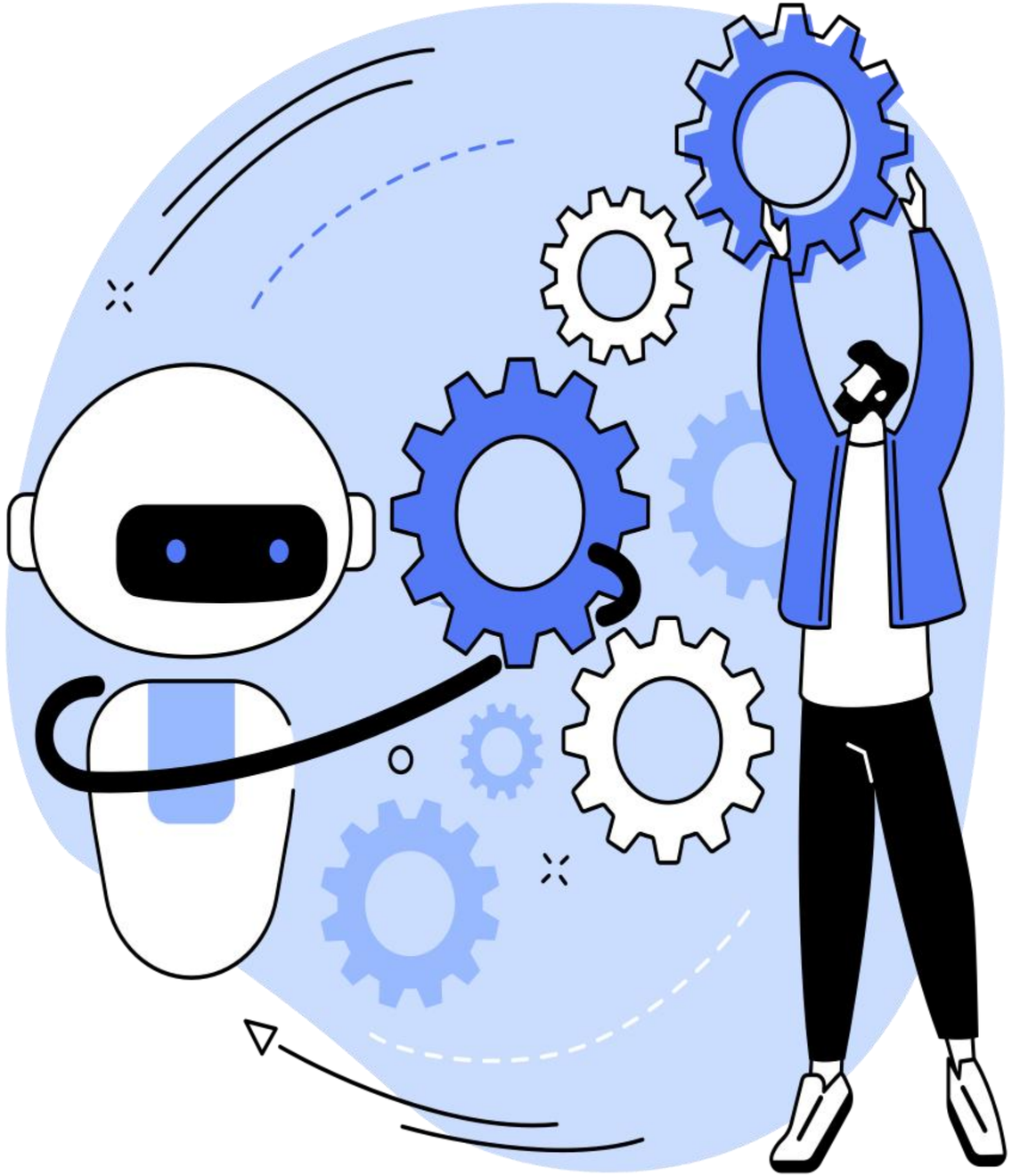




Result

[ElmoGaber/Computational-Neuroscience-2: Assignment (2)]





THANK YOU FOR LISTINING , HOPE YOU ENJOY

Momen Tarek

4221022